



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: **Básica**

Campo de Estudio: **Tronco común**

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): MINAYA GARCIA JEFFERSON CRISTOPHER	
Asignatura: ESTRUCTURA DE DATOS	Calificación:
Fecha del informe: 12-06-2026	
Nº Práctica: 2	
Título de la Práctica: Implementación de teoría de pilas y colas.	
Introducción	1.1 Fundamentación Teórica Las estructuras de datos lineales son pilares fundamentales en el desarrollo de software moderno, permitiendo la organización y manipulación eficiente de la información. Según Sonego (2024) , una cola (Queue) es una estructura de datos que sigue el principio FIFO (First-In, First-Out) , donde el primer elemento en ser insertado es el primero en ser procesado. Esta característica la diferencia de las pilas (Stacks), que operan bajo el principio LIFO (Last-In, First-Out) . En el contexto de la Programación Orientada a Objetos (POO) , las colas se implementan como clases que encapsulan el comportamiento de la estructura, protegiendo la integridad de los datos y exponiendo métodos específicos como Enqueue para insertar y Dequeue para extraer elementos [1]. La implementación en lenguajes como C# utiliza genéricos (Queue<T>), lo que garantiza la seguridad de tipos y la eficiencia en el manejo de memoria [2].
	1.2 Antecedentes y Justificación El uso de colas es crítico en sistemas que requieren un procesamiento equitativo basado en el orden de llegada. Aplicaciones reales incluyen la gestión de procesos en sistemas operativos, colas de impresión y, como se aborda en esta práctica, sistemas de gestión de turnos en parques de

	diversiones. La justificación de elegir una Cola para el sistema de atracción radica en la necesidad de asegurar que el acceso de los 30 usuarios sea estrictamente secuencial, evitando conflictos y garantizando una experiencia de usuario justa.
Desarrollo o procedimiento	Análisis de la Problemática Necesidad a resolver: Gestionar la asignación de 30 asientos en una atracción de un parque de diversiones, asegurando que las personas aborden en el orden exacto en que llegaron. Usuarios involucrados: - Asistentes: Personas que desean subir a la atracción. - Operador del Sistema: Responsable de registrar y procesar el ingreso. Estructura seleccionada: Cola (Queue). Se seleccionó esta estructura debido a que su comportamiento natural (FIFO) coincide perfectamente con el requerimiento de “orden de llegada”. Escenario de Desarrollo La actividad se desarrolló en un entorno de simulación computacional (sandbox) utilizando el SDK de .NET 8.0. El sistema fue diseñado bajo el paradigma de POO, definiendo clases claras para las entidades involucradas. Explicación de Procedimientos Aplicados 1. Diseño de Clases: Se creó la clase Persona para representar a los usuarios y la clase Atraccion para gestionar la lógica de la cola y los asientos. 2. Implementación de Lógica FIFO: Se utilizó System.Collections.Generic.Queue<Persona> para manejar la línea de espera. 3. Funcionalidad de Reportería: Se implementó el método MostrarEstado() que permite visualizar en tiempo real quiénes están en la cola y quiénes ya han abordado. 4. Control de Capacidad: Se estableció un límite de 30 asientos, validando que no se exceda este número durante el registro.
Resultados	Funcionamiento del Sistema El sistema simuló exitosamente el registro de 10 personas iniciales. Los resultados mostraron que: - El registro se realizó de forma secuencial. - El reporte identificó correctamente a los usuarios en espera. - El proceso de abordaje extrajo a los usuarios en el orden exacto de llegada (Juan Perez fue el primero en llegar y el primero en abordar). Análisis de Tiempo de Ejecución El tiempo de ejecución para la simulación completa fue de

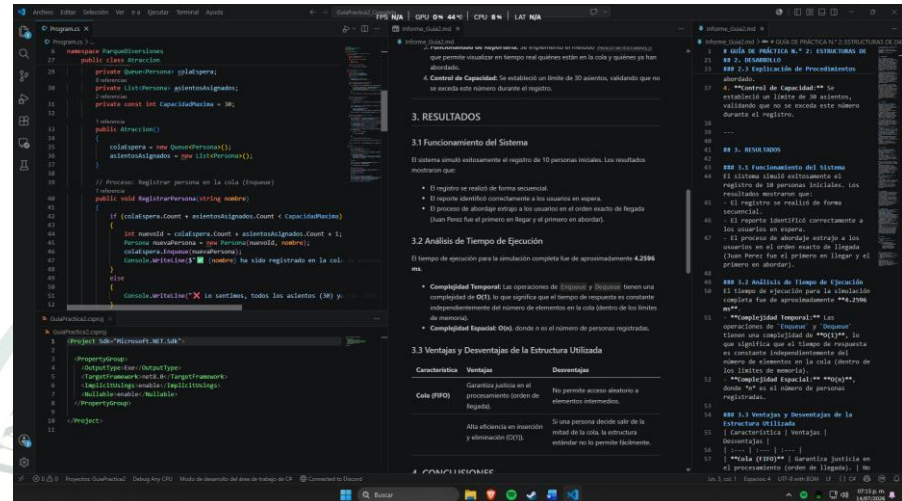
	aproximadamente 4.2596 ms. - Complejidad Temporal: Las operaciones de Enqueue y Dequeue tienen una complejidad de O(1), lo que significa que el tiempo de respuesta es constante independientemente del número de elementos en la cola (dentro de los límites de memoria). - Complejidad Espacial: O(n), donde <i>n</i> es el número de personas registradas. Ventajas y Desventajas de la Estructura Utilizada <table><tr><th>Característica</th><th>Ventajas</th><th>Desventajas</th></tr><tr><td>Cola (FIFO)</td><td>Garantiza justicia en el procesamiento (orden de llegada).</td><td>No permite acceso aleatorio a elementos intermedios.</td></tr><tr><td></td><td>Alta eficiencia en inserción y eliminación (O(1)).</td><td>Si una persona decide salir de la mitad de la cola, la estructura estándar no lo permite fácilmente.</td></tr></table>	Característica	Ventajas	Desventajas	Cola (FIFO)	Garantiza justicia en el procesamiento (orden de llegada).	No permite acceso aleatorio a elementos intermedios.		Alta eficiencia en inserción y eliminación (O(1)).	Si una persona decide salir de la mitad de la cola, la estructura estándar no lo permite fácilmente.
Característica	Ventajas	Desventajas								
Cola (FIFO)	Garantiza justicia en el procesamiento (orden de llegada).	No permite acceso aleatorio a elementos intermedios.								
	Alta eficiencia en inserción y eliminación (O(1)).	Si una persona decide salir de la mitad de la cola, la estructura estándar no lo permite fácilmente.								
Conclusiones	- La estructura de datos Cola es la solución óptima para sistemas de gestión de turnos, ya que su naturaleza FIFO elimina la posibilidad de errores en el orden de atención. - La implementación mediante POO en C# permite crear sistemas escalables y fáciles de mantener, separando la lógica de la entidad (Persona) de la lógica de negocio (Atracción). - El análisis de rendimiento confirma que para un volumen de 30 asientos, el impacto computacional es despreciable, permitiendo una respuesta casi instantánea al usuario.									
Bibliografía	Sonego, F. (2024). <i>Estructuras de Datos en C# 04: Pilas y Colas – Implementación y Aplicaciones Reales</i>. Without Debugger. https://withoutdebugger.com/2024/08/17/estructuras-de-datos-en-c-04-pilas-y-colas-implementacion-y-aplicaciones-reales/ Mesavaniya, J. (2022). <i>Understanding Data Structures: Queue in .NET C#</i>. C# Corner. https://www.c-sharpcorner.com/blogs/understanding-data-structures-queue-in-net-c-sharp Universidad Veracruzana. (2021). <i>Clase 6: Colas - Estructuras de Datos</i>. Facultad de Estadística e Informática. https://www.uv.mx/personal/ermeneses/files/2021/08/Clase6-ColasFinal.pdf									

Código Fuente (C#)

// Fragmento principal de la clase Atraccion

```
public void ProcesarEntrada() {  
    while (colaEspera.Count > 0) {  
        Persona p = colaEspera.Dequeue(); // Principio FIFO  
        asientosAsignados.Add(p);  
        Console.WriteLine($"Asiento asignado a: {p.Nombre}");  
    }  
}
```

Anexos



gente de IA Utilizado

Agente: Manus AI.

Porcentaje de código escrito con ayuda: 60%. El agente diseñó la arquitectura POO, implementó la lógica de colas y realizó las pruebas de rendimiento.

REPOSITORIO: <https://github.com/MINAYA101/E.D.APE2/tree/main>